

Faster index for property matching

Costas S. Iliopoulos¹, M. Sohel Rahman^{*,2,3}

Algorithm Design Group, Department of Computer Science, King's College London, Strand, London WC2R 2LS, England, UK

Received 8 June 2007; received in revised form 3 September 2007; accepted 3 September 2007

Available online 11 September 2007

Communicated by L. Boasson

Abstract

In this paper, we revisit the Property Matching problem studied by Amir et al. [Property Matching and Weighted Matching, CPM 2006] and present a better indexing scheme for the problem. In particular, the data structure by Amir et al., namely PST, requires $O(n \log |\Sigma| + n \log \log n)$ construction time and $O(m \log |\Sigma| + K)$ query time, where n and m are the length of, respectively, the text and the pattern, Σ is the alphabet and K is the output size. On the other hand, the construction time of our data structure, namely IDS_PIP, is dominated by the suffix tree construction time and hence is $O(n)$ time for alphabets that are natural numbers from 1 to a polynomial in n and $O(n \log \sigma)$ time otherwise, where $\sigma = \min(n, |\Sigma|)$. The query time is same as that of PST. Also, IDS_PIP has the advantage that it can be built on either a suffix tree or a suffix array and additionally, it retains the capability of answering normal pattern matching queries.

© 2007 Elsevier B.V. All rights reserved.

Keywords: Algorithms; Combinatorial problems; Data structures; Pattern matching

1. Introduction

In this paper, we deal with the problem of Pattern Matching with Properties (Property Matching, for short) [2,3], which is an interesting variant of the classic pattern matching problem. In the classical pattern matching problem, the goal is to find all the occurrences

of a given pattern $P = P[1..m]$ of length m in a text $T = T[1..n]$ of length n , both being sequences of characters drawn from a finite character set Σ . In property matching problem, we have the added constraint that the particular substring in the text matching with the pattern must satisfy some property. This problem is motivated by practical applications as follows. In many text search situations, it may be the case that only certain parts of a huge text collection are important for the search. Also, in molecular biology, it has long been a practice to consider special genome areas by their structures. Examples are repetitive genomic structures [8] such as tandem repeats, LINEs (Long Interspersed Nuclear Sequences) and SINEs (Short Interspersed Nuclear Sequences) [9]. In this case, the task is to find occurrences of a given pattern in a genome, provided it appears in a SINE, or LINE [2,3].

* Corresponding author.

E-mail addresses: csi@dcs.kcl.ac.uk (C.S. Iliopoulos), sohel@dcs.kcl.ac.uk (M.S. Rahman).

URL: <http://www.dcs.kcl.ac.uk/adg>.

¹ Supported by EPSRC and Royal Society grants.

² Supported by the Commonwealth Scholarship Commission in the UK under the Commonwealth Scholarship and Fellowship Plan (CSFP).

³ On Leave from Department of CSE, BUET, Dhaka-1000, Bangladesh.

Property Matching was investigated in a very recent paper of Amir et al. [2,3]. As was pointed out in [2,3], the ‘sequential’ version of the problem has a straightforward linear time solution. However, the indexing version of the problem is not straightforward at all. In [2,3], the authors presented an index data structure, namely Property Suffix Tree (PST), essentially a modification of the suffix tree, and achieved ‘almost’ the same bounds that exist in the literature for ordinary indexing. In particular, the construction time for PST is $O(n \log |\Sigma| + n \log \log n)$ and the corresponding query time is $O(m \log |\Sigma| + K)$, where K is the output size.

In this paper, we revisit the Property Matching problem and present a better index for the problem. The advantages of our data structure are threefold:

1. Firstly, our data structure construction time is same as that of a suffix tree (or a suffix array) and hence is better than that of PST.
2. Secondly, we can construct our data structure using either the suffix tree or the suffix array. This gives us the freedom of choice in application specific cases. For example, when space complexity is important, we can use the suffix array instead of the suffix tree.
3. Thirdly, unlike PST, we can use our data structure for normal pattern matching as well. As a result, we can perform normal pattern matching queries as well as queries for property matching with the same data structure, which may turn out to be useful in different complex applications.

The rest of the paper is organized as follows. In Section 2, we present the preliminary concepts. The new data structure for property matching is presented in Section 3. We conclude briefly in Section 4.

2. Preliminaries

A *text*, also called a *string*, is a sequence of zero or more symbols from an alphabet Σ . A text T of length n is denoted by $T[1..n] = T_1 T_2 \dots T_n$, where $T_i \in \Sigma$ for $1 \leq i \leq n$. The *length* of T is denoted by $|T| = n$. A string w is a *factor* or *substring* of T if $T = uwv$ for $u, v \in \Sigma^*$; in this case, the string w occurs at position $|u| + 1$ in T . The factor w is denoted by $T[|u| + 1..|u| + |w|]$. A *prefix* (*suffix*) of T is a factor $T[x..y]$ such that $x = 1$ ($y = n$), $1 \leq y \leq n$ ($1 \leq x \leq n$).

In traditional pattern matching problem, we want to find the occurrences of a given pattern $P[1..m]$ in a text $T[1..n]$. The pattern P is said to occur at position $i \in [1..n]$ of T if and only if $P = T[i..i + m - 1]$. We use Occ_T^P to denote the set of occurrences of P

in T . Now, we present formal definitions of the Property Matching problem and related concepts. The definitions are taken from [2,3] after slight adaptation to match our notations.

Definition 1. A property π of a string $T[1..n] = T_1 T_2 \dots T_n$ is a set of intervals $\pi = \{(s_1, f_1), \dots, (s_{|\pi|}, f_{|\pi|})\}$, where for each $1 \leq i \leq |\pi|$, it holds that $s_i, f_i \in [1..n]$ and $s_i \leq f_i$. The size of the property π , denoted by $|\pi|$ is the number of intervals in the property.

As in [2,3], we assume that the property information is given in the standard form as defined below.

Definition 2. A property π for a string of length n is said to be in standard form if we have $s_1 < s_2 < \dots < s_{|\pi|}$. Note that, this will also mean that, for any $1 \leq i \leq n$, there is at most one $(s_k, f_k) \in \pi$ such that $s_k = i$.

We are interested in the indexing version of the Property Matching problem which is defined formally below.

Problem “PIP” (Property Indexing Problem). Suppose we are given a text $T = T_1 \dots T_n$ with property $\pi = \{(s_1..f_1), (s_2..f_2), \dots, (s_{|\pi|}..f_{|\pi|})\}$. Preprocess T to answer the following form of queries.

Query: Given a pattern $P = P_1 \dots P_m$, construct the set:

$$Occ_{T,\pi}^P = \{i \mid P = T[i..i + m - 1] \wedge \exists (s_k, f_k) \in \pi : (s_k \leq i \leq i + m - 1 \leq f_k)\}.$$

Since our data structure is built on top of a suffix tree or a suffix array, we give a very brief account of them as follows. Given a string T of length n over an alphabet Σ , the suffix tree ST_T of T is the compacted trie of all suffixes of $T\$$, where $\$ \notin \Sigma$. Each leaf in ST_T represents a suffix $T[i..n]$ of T and is labeled with the index i . We refer to the list (in left-to-right order) of indices of the leaves of the subtree rooted at node v as the leaf-list of v ; it is denoted by $LL(v)$. Each edge in ST_T is labeled with a nonempty substring of T such that the path from the root to the leaf labeled with index i spells the suffix $T[i..n]$. For any node v , we let ℓ_v denote the string obtained by concatenating the substrings labeling the edges on the path from the root to v in the order they appear. Several algorithms exist that can construct the suffix tree ST_T requiring linear space in $O(n \log \sigma)$ time, where $\sigma = \min(n, |\Sigma|)$ [14, 16]. Notably, this means that, for bounded alphabet the construction time is $O(n)$. Furthermore, in [6], Farach

presented an $O(n)$ time algorithm for suffix tree construction, which works even for larger alphabets. In particular, Farach's algorithm works in linear time even if we have $\Sigma = \{1, \dots, n^c\}$, where c is a constant. Given the suffix tree $ST_{\mathcal{T}}$ of a text $\mathcal{T}$ we define the "locus" $\mu^{\mathcal{P}}$ of a pattern $\mathcal{P}$ as the node in $ST_{\mathcal{T}}$ such that $\ell_{\mu^{\mathcal{P}}}$ has the prefix $\mathcal{P}$ and $|\ell_{\mu^{\mathcal{P}}}|$ is the smallest of all such nodes. Note that the locus of $\mathcal{P}$ does not exist, if $\mathcal{P}$ is not a substring of $\mathcal{T}$. Therefore, given $\mathcal{P}$, finding $\mu^{\mathcal{P}}$ suffices to determine whether $\mathcal{P}$ occurs in $\mathcal{T}$. Given a suffix tree of a text $\mathcal{T}$ and a pattern $\mathcal{P}$, one can find its locus and hence the fact whether $\mathcal{T}$ has an occurrence of $\mathcal{P}$ in $O(|\mathcal{P}| \log |\Sigma|)$ time. In addition to that, all such occurrences can be reported in constant time per occurrence. Finally, we note that, optimal query time of $O(|\mathcal{P}| + Occ_{\mathcal{T}}^{\mathcal{P}})$ can be achieved with suffix tree with $O(n \log n)$ bits of space [4].

We end this section with a very concise definition of suffix array. The suffix array $SA[1..n]$ of a text $\mathcal{T}$ is an array of integers $j \in [1..n]$ such that $SA[i] = j$ if, and only if, $\mathcal{T}[j..n]$ is the i th suffix of $\mathcal{T}$ in (ascending) lexicographic order. Suffix arrays were first introduced in [13], where an $O(n \log n)$ construction algorithm and $O(m + \log n + |Occ_{\mathcal{T}}^{\mathcal{P}}|)$ query time were presented. Recently, linear time construction algorithms for space efficient suffix arrays have been presented [10–12]. The query time is also improved to optimal $O(m + |Occ_{\mathcal{T}}^{\mathcal{P}}|)$ in [1]. We recall that, the result of a query for a pattern $\mathcal{P}$ on a suffix array SA of $\mathcal{T}$, is given in the form of an interval $[s..e]$ such that $Occ_{\mathcal{T}}^{\mathcal{P}} = \{SA[s], SA[s+1], \dots, SA[e]\}$. In this case, the interval $[s..e]$ is denoted by $Int_{\mathcal{T}}^{\mathcal{P}}$.

3. An index for problem PIP

In this section, we present our new data structure to solve problem PIP. Our basic idea is to build an index data structure that would solve the problem in two steps. First, it will (implicitly) give us the set $Occ_{\mathcal{T}}^{\mathcal{P}}$. Then, the index would work as a filter and 'select' some of the occurrences to provide us with our desired set $Occ_{\mathcal{T}, \pi}^{\mathcal{P}}$. We describe now the idea we employ. We first construct a suffix tree $ST_{\mathcal{T}}$ of $\mathcal{T}$. According to the definition of the suffix tree, each leaf in $ST_{\mathcal{T}}$ is labeled by the starting location of its suffix. We do some preprocessing on $ST_{\mathcal{T}}$ as follows. We maintain a linked list of all leaves in a left-to-right order. In other words, we realize the list $LL(\mathcal{R})$ in the form of a linked list, where $\mathcal{R}$ is the root of the suffix tree. Also, we set pointers $v.left$ and $v.right$ from each tree node v to its leftmost leaf v_{ℓ} and rightmost leaf v_r (considering the subtree rooted at v) in the linked list. It is easy to realize that, with

these set of pointers at our disposal, we can indicate the set of occurrences of a pattern $\mathcal{P}$ in $\mathcal{T}$ by the two leaves $\mu_{\ell}^{\mathcal{P}}$ and $\mu_r^{\mathcal{P}}$, because all the leaves between and including $\mu_{\ell}^{\mathcal{P}}$ and $\mu_r^{\mathcal{P}}$ in $LL(\mathcal{R})$ correspond to the occurrences of $\mathcal{P}$ in $\mathcal{T}$. We construct an array $\mathcal{L}$ realizing the list $LL(\mathcal{R})$.

Now, recall that our data structure has to be able to somehow "select" and report only those occurrences that is completely contained in one of the intervals in π . To achieve that, we use the following interesting problem.

Problem "RMAX" (Range Maxima Query Problem).

We are given an array $B[1..n]$ of numbers. We need to preprocess B to answer the following form of queries:

Query: Given an interval $I = (i_s..i_e)$, $1 \leq i_s \leq i_e \leq n$, the goal is to find the index k (or the value $B[k]$ itself) with maximum value $B[k]$ for $k \in I$.

Problem RMAX has received much attention in the literature and Bender and Farach-Colton showed that we can build a data structure in $O(n)$ time using $O(n \log n)$ -bit space and can answer subsequent queries in $O(1)$ time per query [5].⁴ Recently, Sadakane [15] presented a succinct data structure which achieves the same time complexity using $O(n)$ bits of space.

Now, we perform the following steps. We assume that the intervals of π are disjoint. Otherwise, we can always create an 'equivalent' set of intervals π' , in $O(n)$ time such that the intervals of π' are disjoint. This is done as follows. Recall that π is in standard form (Definition 2). We start with initializing π' to $\emptyset$. Now, consider an interval $e_k \equiv (s_k, f_k) \in \pi$. We check $e_{k+1} \equiv (s_{k+1}, f_{k+1}) \in \pi$ against e_k as follows. If $s_{k+1} > f_k$, then the two intervals are disjoint; so we include e_k in π' and continue with e_{k+1} , i.e. start checking e_{k+2} against e_{k+1} . If, on the other hand, $s_{k+1} \leq f_k$, i.e., the intervals are not disjoint, then we create an 'equivalent' interval $e' \equiv (s', f')$ as follows. If $f_{k+1} \leq f_k$ (i.e. e_{k+1} is contained in e_k), then we assign $e' = e_k$, otherwise we assign, $e' \equiv (s_k, f_{k+1})$ and continue to check the next interval in π against e' . It is easy to see that, the above procedure works in $O(n)$ and the resulting set π' is a set of disjoint intervals 'equivalent' to π .

Now, assuming π is disjoint, we continue as follows. We maintain an array $\mathcal{A}[1..n]$ such that each $\mathcal{A}[i] \equiv (p, q)$ is a two tuple. For $1 \leq i \leq n$, $\mathcal{A}[i]$ is defined as follows:

⁴ The same result was achieved in [7], albeit with a more complex data structure.

$$\mathcal{A}[i].p = \begin{cases} f_k - \mathcal{L}[i] + 1 & \text{if } s_k \leq \mathcal{L}[i] \leq f_k \\ -\mathcal{L}[i] & \text{otherwise} \end{cases} \quad (1)$$

$$\mathcal{A}[i].q = \begin{cases} f_k - s_k + 1 & \text{if } s_k \leq \mathcal{L}[i] \leq f_k \\ -1 & \text{otherwise.} \end{cases} \quad (2)$$

We now define the relation $\succ$, $\prec$ and $\doteq$ on $\mathcal{A}[i]$, $1 \leq i \leq n$ as follows. We say that $\mathcal{A}[i] \doteq \mathcal{A}[j]$, $1 \leq i, j \leq n$, if, and only if, $i = j$. So, there exists no i, j , $1 \leq i \neq j \leq n$ such that $\mathcal{A}[i] \doteq \mathcal{A}[j]$. On the other hand, we say that $\mathcal{A}[i] \succ \mathcal{A}[j]$ (resp. $\mathcal{A}[i] \prec \mathcal{A}[j]$), $1 \leq i \neq j \leq n$, if, and only if, any of the following is true:

1. $\mathcal{A}[i].p > \mathcal{A}[j].p$ (resp. $\mathcal{A}[i].p < \mathcal{A}[j].p$),
2. $\mathcal{A}[i].p = \mathcal{A}[j].p \wedge \mathcal{A}[i].q > \mathcal{A}[j].q$ (resp. $\mathcal{A}[i].q < \mathcal{A}[j].q$),
3. $\mathcal{A}[i].p = \mathcal{A}[j].p \wedge \mathcal{A}[i].q = \mathcal{A}[j].q \wedge i > j$ (resp. $i < j$).

Now, to complete the construction of the data structure, we preprocess the array $\mathcal{A}[i]$ for range maxima query realizing the relations $\succ$, $\prec$ and $\doteq$. In the rest of this paper, we refer to this data structure as IDS_PIP.

3.1. Analysis

Let us now analyze the cost of building the index data structure IDS_PIP. To build IDS_PIP, we first construct a traditional suffix tree $ST_{\mathcal{T}}$ requiring $O(n)$ time for alphabets that are natural numbers from 1 to a polynomial in n and $O(n \log \sigma)$ time otherwise. The preprocessing on the suffix tree can be done in $O(n)$ by traversing $ST_{\mathcal{T}}$ using a breadth first or an in order traversal. If π is not disjoint, we can get an equivalent disjoint set of intervals π' spending $O(n)$ time. We now turn our attention to the construction time of the array $\mathcal{A}$. We first realize the relation $\mathcal{V}$ such that $\mathcal{V}[i] = (s_k, f_k)$, $1 \leq i \leq n$, if, and only if, $(s_k, f_k) \in \pi$ and $s_k \leq \mathcal{L}[i] \leq f_k$. We also realize the inverse relation (of $\mathcal{L}$) $\mathcal{L}^{-1}[i]$, $1 \leq i \leq n$, such that $\mathcal{L}^{-1}[k] = i \Leftrightarrow \mathcal{L}[i] = k$. $\mathcal{L}^{-1}$ can be realized simply by scanning $\mathcal{L}$ and with $\mathcal{L}^{-1}$ in our hand, we can realize $\mathcal{V}$ in $O(n)$ time, because, π can be assumed to be disjoint. With $\mathcal{V}$ and $\mathcal{L}$ at our disposal, it is straightforward to construct the array $\mathcal{A}$ in $O(n)$ time. Finally, we preprocess array $\mathcal{A}$ for range maxima queries requiring $O(n)$ time. Therefore, the total construction time is dominated by the suffix tree construction time and is $O(n)$ time for alphabets that are natural numbers from 1 to a polynomial in n and $O(n \log \sigma)$ time otherwise. The space requirement is linear.

-
- 1: Find $\mu^{\mathcal{P}}$ in $ST_{\mathcal{T}}$
 - 2: Set $i = \mu^{\mathcal{P}}.left$, $j = \mu^{\mathcal{P}}.right$
 - 3: $Occ_{\mathcal{T}, \pi}^{\mathcal{P}} = \epsilon$
 - 4: $FindOccurrence(\mathcal{L}, i, j, |\mathcal{P}|)$ {See Algorithm 2}
-

Algorithm 1. Algorithm for query processing.

-
- 1: $k = RangeMaximaQuery(\mathcal{A}, i, j)$
 - 2: **if** $\mathcal{A}[k].p \geq m$ **then**
 - 3: Set $Occ_{\mathcal{T}, \pi}^{\mathcal{P}} = Occ_{\mathcal{T}, \pi}^{\mathcal{P}} \cup \mathcal{L}[k]$
 - 4: $FindOccurrence(\mathcal{L}, i, k - 1, m)$
 - 5: $FindOccurrence(\mathcal{L}, k + 1, j, m)$
 - 6: **end if**
-

Algorithm 2. Procedure $FindOccurrence(\mathcal{L}, i, j, m)$.

3.2. Query processing

So far we have concentrated on the construction of IDS_PIP. Now we discuss the query processing. Suppose we are given a query pattern $\mathcal{P}$. We first find the locus $\mu^{\mathcal{P}}$ in $ST_{\mathcal{T}}$. Let $i = \mu^{\mathcal{P}}.left$ and $j = \mu^{\mathcal{P}}.right$. This means that we get the set $Occ_{\mathcal{T}}^{\mathcal{P}}$ in the form of $\mathcal{L}[i..j]$. Now, we apply a divide and conquer approach as follows. We perform a Range Maxima Query on $\mathcal{A}$ in the interval $[i..j]$. Suppose, the query returns the index k . If $\mathcal{A}[k].p \geq m$, then we put $\mathcal{L}[k]$ in $Occ_{\mathcal{T}, \pi}^{\mathcal{P}}$ and then, we (recursively) perform the range maxima queries in the intervals $[i..k - 1]$ and $[k + 1..j]$ and continue as before. If any of the queries returns k such that $\mathcal{A}[k].p < m$ we stop. The steps for the query processing are formally stated in Algorithms 1 and 2.

The running time of the query processing is deduced as follows. Finding the locus $\mu^{\mathcal{P}}$ requires $O(m \log |\Sigma|)$ time. The corresponding pointers can be found in constant time. Now, each range maxima query requires $O(1)$ time. And note that, in this way, for each found occurrence in $Occ_{\mathcal{T}, \pi}^{\mathcal{P}}$, we have at most 2 intervals to perform range maxima queries further. These 2 queries, either give new occurrences or stop, ensuring constant time work per occurrence. So, in total the time spent doing the range maxima queries is $O(|Occ_{\mathcal{T}, \pi}^{\mathcal{P}}|)$.

Next, we discuss the correctness of our procedures. We start with the following facts and lemmas.

Fact 1. For all $1 \leq k \leq n$, we have $\mathcal{A}[k].p > 0$ if, and only if, there exists an interval $(s_{\ell}, f_{\ell}) \in \pi$ such that $s_{\ell} \leq \mathcal{L}[k] \leq f_{\ell}$. Same applies for $\mathcal{A}[k].q$.

Fact 2. Suppose, $(s_{\ell}, f_{\ell}) \in \pi$ and $s_{\ell} \leq \mathcal{L}[k] \leq f_{\ell}$. We have $\mathcal{A}[k].p \geq r$ if, and only if, $f_{\ell} - \mathcal{L}[k] + 1 \geq r$.

Lemma 1. Suppose, $\mathcal{L}[k] \in \text{Occ}_T^{\mathcal{P}}$. We have $\mathcal{L}[k] \in \text{Occ}_{T,\pi}^{\mathcal{P}}$ if, and only if, $A[k].p \geq m$, where $|\mathcal{P}| = m$.

Proof. We first proof the ‘if’ part and then the ‘only if’ part.

- $\Rightarrow$: If $A[k].p \geq m (> 0)$, then according to Fact 1, there must exists a $(s_\ell, f_\ell) \in \pi$ such that $s_\ell \leq \mathcal{L}[k] \leq f_\ell$. And, according to Fact 2, we must have $f_\ell - \mathcal{L}[k] + 1 \geq m$. Therefore, we must have $\mathcal{L}[k] \in \text{Occ}_{T,\pi}^{\mathcal{P}}$.
- $\Leftarrow$: Similar to above arguments, since, Facts 1 and 2 are both necessary and sufficient. $\square$

Lemma 2. Suppose, $\mathcal{L}[k_1], \mathcal{L}[k_2] \in \text{Occ}_T^{\mathcal{P}}$ and $A[k_1] < A[k_2]$. If $\mathcal{L}[k_1] \in \text{Occ}_{T,\pi}^{\mathcal{P}}$ then we must also have $\mathcal{L}[k_2] \in \text{Occ}_{T,\pi}^{\mathcal{P}}$.

Proof. Since, $\mathcal{L}[k_1] \in \text{Occ}_{T,\pi}^{\mathcal{P}}$, according to Lemma 1, we must have $A[k_1].p \geq m$. Now, according to definition, if $A[k_1] < A[k_2]$, we must have $A[k_1].p \leq A[k_2].p$. Therefore, according to assumption, we have $A[k_2].p \geq m$. Therefore, it follows from Lemma 1 that $\mathcal{L}[k_2] \in \text{Occ}_{T,\pi}^{\mathcal{P}}$. $\square$

Lemma 3. Algorithm 1 correctly computes the set $\text{Occ}_{T,\pi}^{\mathcal{P}}$.

Proof. Immediate from Lemmas 1 and 2. $\square$

The result of this section is summarized in the form of following theorem.

Theorem 1. For Problem PIP, we can construct the IDS_PIP data structure requiring $O(n)$ space in $O(n)$ time for alphabets that are natural numbers from 1 to a polynomial in n and $O(n \log \sigma)$ time otherwise, where $\sigma = \min(n, |\Sigma|)$. Then, we can answer the relevant queries in $O(m \log |\Sigma| + |\text{Occ}_{T,\pi}^{\mathcal{P}}|)$ time per query.

3.3. Further remarks

There exist algorithms for suffix tree construction requiring $O(n \log n)$ bits of space, which can answer the queries in optimal $O(m + |\text{Occ}_{T,\pi}^{\mathcal{P}}|)$ time [4]. Therefore, we get the following extension of Theorem 1.

Theorem 2. For Problem PIP, we can construct the IDS_PIP data structure requiring $O(n \log n)$ bits of space in $O(n)$ time for alphabets that are natural numbers from 1 to a polynomial in n and $O(n \log \sigma)$ time

otherwise, where $\sigma = \min(n, |\Sigma|)$. Then, we can answer the relevant queries in optimal $O(m + |\text{Occ}_{T,\pi}^{\mathcal{P}}|)$ time per query.

We remark that, IDS_PIP can also be built on the classic suffix array data structure instead of the suffix tree. This may turn out to be beneficial in certain applications, because, although both data structure requires same space asymptotically, in reality, suffix arrays are space economical than suffix trees. It is easy to realize that, while constructing IDS_PIP using suffix array, we just need to consider the array SA instead of the array $\mathcal{L}$.

Another interesting thing about our data structure is that, unlike PST [2,3], IDS_PIP can serve dual purposes in the sense that, it can still answer queries of the classic pattern matching problem. This follows from the fact that the suffix tree (or suffix array) remains unchanged as the base of IDS_PIP while, in case of PST, some information from the suffix tree are deleted. We believe that, this unique characteristic of our data structure may turn out to be useful in different complex applications having multiple objectives.

Finally, in [2,3], it was shown how the (indexed) Weighted Pattern Matching problem (IWPM) can be solved using the property matching problem. For definitions of IWPM, the reduction to PIP, and the relevant details we refer to [2,3]. We just mention that, using our data structure, naturally, we get better running time for IWPM as well.

4. Conclusion

In this paper, we have revisited the Property Matching problem which was studied in [2,3] and have presented a better indexing scheme for it. In particular, the data structure in [2,3], namely PST, requires $O(n \log |\Sigma| + n \log \log n)$ construction time and $O(m \log |\Sigma| + |\text{Occ}_{T,\pi}^{\mathcal{P}}|)$ query time. Our data structure, IDS_PIP, on the other hand, exhibits same query time as does PST, but has a better construction time. In particular, the construction time of IDS_PIP is dominated by suffix tree construction time and hence is $O(n)$ time for alphabets that are natural numbers from 1 to a polynomial in n and $O(n \log \sigma)$ time otherwise. The space requirement is linear as in PST. Furthermore, we can shave off the $\log |\Sigma|$ factor in the query time and make it optimal, if we are allowed $O(n \log n)$ bits of space for the construction of the suffix tree. We believe that, apart from exhibiting a better running time, IDS_PIP has a number of advantages over PST as follows.

1. IDS_PIP can be constructed using either the suffix tree or the suffix array data structure. This gives us the freedom of choice in application specific cases. For example, when space complexity is important, we can use the suffix array instead of suffix tree.
2. Unlike PST, IDS_PIP can be used for normal pattern matching as well. As a result, we can perform normal pattern matching queries as well as queries for property matching with the same data structure.

Acknowledgement

The authors would like to express their gratitude to the anonymous reviewers for their helpful suggestions.

References

- [1] M.I. Abouelhoda, E. Ohlebusch, S. Kurtz, Optimal exact string matching based on suffix arrays, in: A.H.F. Laender, A.L. Oliveira (Eds.), *SPIRE*, in: *Lecture Notes in Computer Science*, vol. 2476, Springer, 2002, pp. 31–43.
- [2] A. Amir, E. Chencinski, C.S. Iliopoulos, T. Kopelowitz, H. Zhang, Property matching and weighted matching, in: M. Lewenstein, G. Valiente (Eds.), *CPM*, in: *Lecture Notes in Computer Science*, vol. 4009, Springer, 2006, pp. 188–199.
- [3] A. Amir, E. Chencinski, C.S. Iliopoulos, T. Kopelowitz, H. Zhang, Property matching and weighted matching, *Theor. Comput. Sci.* (2007), in press.
- [4] A. Apostolico, The myriad virtues of subword trees, in: *Combinatorial Algorithms on Words*, in: *NATO ISI Series*, Springer, 1985, pp. 85–96.
- [5] M.A. Bender, M. Farach-Colton, The lca problem revisited, in: *Latin American Theoretical Informatics (LATIN)*, 2000, pp. 88–94.
- [6] M. Farach, Optimal suffix tree construction with large alphabets, in: *FOCS*, 1997, pp. 137–143.
- [7] H. Gabow, J. Bentley, R. Tarjan, Scaling and related techniques for geometry problems, in: *Symposium on the Theory of Computing (STOC)*, 1984, pp. 135–143.
- [8] J. Jurka, Human repetitive elements, in: R.A. Meyers (Ed.), *Molecular Biology and Biotechnology*.
- [9] J. Jurka, Origin and evolution of alu repetitive elements, in: R. Marais (Ed.), *The Impact of Short Interspersed Elements (SINEs) on the Host Genome*.
- [10] J. Kärkkäinen, P. Sanders, S. Burkhardt, Simple linear work suffix array construction, *J. ACM* 53 (6) (2006) 918–936.
- [11] D.K. Kim, J.S. Sim, H. Park, K. Park, Constructing suffix arrays in linear time, *J. Discrete Algorithms* 3 (2–4) (2005) 126–142.
- [12] P. Ko, S. Aluru, Space efficient linear time construction of suffix arrays, *J. Discrete Algorithms* 3 (2–4) (2005) 143–156.
- [13] U. Manber, E.W. Myers, Suffix arrays: A new method for on-line string searches, *SIAM J. Comput.* 22 (5) (1993) 935–948.
- [14] E.M. McCreight, A space-economical suffix tree construction algorithm, *J. ACM* 23 (2) (1976) 262–272.
- [15] K. Sadakane, Succinct data structures for flexible text retrieval systems, *J. Discrete Algorithms* 5 (1) (2007) 12–22.
- [16] E. Ukkonen, On-line construction of suffix trees, *Algorithmica* 14 (3) (1995) 249–260.